

ivv-recon

Contents

ivv-recon	1
Why this exists	1
What it checks	2
The hard part: canonicalization	2
Install	3
Try it — no database needed	3
Real use	3
Mapping file	4
The runbook	4
Development	5
Background	5
License	5

ivv-recon

Source-to-target reconciliation for ETL migrations

Ali Sarour · asarour@gmail.com · github.com/asarour/ivv-recon · linkedin.com/in/ali-sarour

Source-to-target reconciliation for ETL migrations. Proves that the data which left a source system arrived intact in a target — across engines that cannot see each other.

Supports **SQL Server**, **Azure SQL**, **Azure Synapse**, and **Snowflake**.

Why this exists

Most migration “validation” is a row count and a spot check on a few tables. That catches the failures nobody was worried about and misses the ones that end careers.

Here is the demo dataset that ships with this repo:

```
[PASS] row_counts          0 finding(s)
```

Seven rows in the source, seven rows in the target. Counts reconcile perfectly.

They are not the same seven rows. One customer never arrived, and a phantom row took its place in the total. A count check signs that off. Eighteen months later it surfaces in an audit.

```
[FAIL] key_sets            2 finding(s)
      CRITICAL 1 keys present in source but missing from target
      HIGH    1 keys present in target but not in source
```

That is the whole argument for this tool.

What it checks

#	Check	Catches
1	Schema types	Silent truncation, relaxed or tightened nullability
2	Row counts	Gross data loss or double-loads
3	Key integrity	Non-unique or NULL keys — which invalidate everything downstream
4	Key sets	Rows that vanished, rows that appeared (the EXCEPT / MINUS)
5	Row hashes	Content divergence on rows present in both
6	Field-level	Which <i>column</i> is responsible for the mismatches
7	NULL handling	NULLs silently replaced by ' ' or 0 via a DEFAULT
8	Aggregates	An independent cross-check that the hash path is not lying

Checks run cheap-to-expensive and gate each other. If the declared key is not unique, row-level comparison is **skipped** rather than reported — with a broken key, every row-level finding is noise, and noise is how real findings get ignored.

The hard part: canonicalization

A row in SQL Server and “the same” row in Snowflake **will not** produce the same hash. Naive comparison reports 100% mismatch and gets abandoned on day one.

Every rule below exists because one of these produced a false positive — or worse, a false negative — on a real migration:

Problem	Rule
CHAR(20) right-pads with spaces; Snowflake VARCHAR does not	RTRIM both sides. Never LTRIM — leading space is data
DECIMAL(18,2) renders 100.50;	Coerce both to fixed scale 10
NUMBER(38,10) renders 100.5000000000	
NUMBER(38,0) is Snowflake’s landing type for INT — comparing them as different types means no key ever matches	Parse scale; scale 0 is an integer
SQL Server DATETIME holds ~3.33 ms;	Truncate to milliseconds — never round, which can roll into the next second
TIMESTAMP_NTZ holds ns	Fold to lowercase
HASHBYTES + CONVERT(...,2) returns UPPERCASE hex; SHA2() returns lowercase	
BIT vs BOOLEAN	Normalize to '1' / '0'
NULL \ \ 'x' is NULL — an entire row collapses to NULL, and two such rows compare <i>equal</i>	Substitute a sentinel before concatenating
('ab', 'c') and ('a', 'bc') concatenate identically	Separate fields with CHAR(31)

Problem	Rule
Synapse caps HASHBYTES at 8000 bytes — wide rows hash their first 8 KB and falsely match	Refuse pushdown; hash in Python

That NULL row is the dangerous one. It is a **false negative**: broken data reported as clean. Every other bug in this table wastes a day. That one ships.

Both the SQL and the Python implementations of these rules are pinned by tests — see [tests/test_canonical.py](#).

Install

```
git clone https://github.com/asarour/ivv-recon.git
cd ivv-recon
pip install -e ".[dev]"
```

Try it — no database needed

```
ivv-recon demo --out ./out
```

Runs the full engine against an in-memory dataset with seven seeded defects, and writes `report.html`, `report.json`, and `runbook.md`. Takes about four seconds.

The demo deliberately includes **traps** — a CHAR(20) padded surname, a 100.5 vs 100.50 balance, a sub-millisecond timestamp. These rows are identical in substance. If the tool reports any of them, canonicalization is broken and it is worse than useless, because it sends an engineer chasing ghosts on every run. [tests/test_engine.py](#) asserts they stay quiet.

Real use

```
ivv-recon validate --config mapping.yml    # parse-check the mapping, no DB
ivv-recon runbook  --config mapping.yml    # generate the migration runbook
```

```
from ivv_recon.config import load_spec
from ivv_recon.connectors import SQLConnector
from ivv_recon.dialects import get_dialect
from ivv_recon.engine import ReconEngine
from ivv_recon.report import write_html

spec = load_spec("mapping.yml")

source = SQLConnector(pyodbc.connect(...), get_dialect("sqlserver"))
target = SQLConnector(snowflake.connector.connect(...), get_dialect("snowflake"))

report = ReconEngine(source, target, spec).run()
write_html(report, "report.html")
raise SystemExit(report.exit_code())
```

Connections are injected; this library never sees a credential. That keeps secrets out of the config file, out of tracebacks, and out of version control.

Exit codes: 0 pass · 1 findings at or above threshold · 2 a check errored.

2 is not 1. A check that raised means we do **not know** whether the data is good. Unknown is not clean, and it never passes.

Scale

Only the key and a 64-character hash cross the wire — never the row. A 40M-row, 200-column table costs megabytes to reconcile, not gigabytes. Hashing is pushed into the database wherever the platform can do it correctly.

Mapping file

```
source:
  dialect: sqlserver
  database: CoreBank
  schema: dbo
  table: CUSTOMER

target:
  dialect: snowflake
  database: ANALYTICS
  schema: RAW
  table: CUSTOMER

keys:
  - source: {name: CUSTOMER_ID, type: INT, nullable: false}
    target: {name: CUSTOMER_ID, type: NUMBER(38,0), nullable: false}

compare:
  - name: EMAIL
    type: VARCHAR(100)
  - source: {name: BALANCE, type: DECIMAL(18,2)}
    target: {name: BALANCE, type: NUMBER(38,10)}
    note: widened; both sides canonicalized to scale 10

exclude:
  - LOAD_TIMESTAMP      # written by the load itself; will never match
```

Full example with every option: [config/example.yml](#).

Unmapped native types **raise**. Defaulting an unknown type to STRING is how you end up hashing a BLOB as text and reporting clean.

The runbook

ivv-recon runbook generates the document an auditor asks for and nobody has written: scope, the full source-to-target mapping table, type conversions needing sign-off, the validation procedure, a three-run mock migration plan with exit criteria, cutover steps, **rollback triggers**, and a sign-off block.

It is generated from the same spec as the report, so it cannot drift from what was actually validated. Excluded columns are printed explicitly — an excluded column is a hole in the assurance and the auditor deserves to see it.

Development

```
pytest # 70 tests, no database required
ruff check src tests
```

The reconciliation logic is pure functions over already-fetched data — no I/O. That is why the suite runs in CI in seconds against three Python versions, and why InMemoryConnector and SQLConnector are interchangeable.

Background

Built by Ali Sarour, who spent two years leading IV&V validation across 1,000+ IBM DataStage ETL jobs for the New York State DMV modernization — reconciling Oracle, DB2, SQL Server and Mainframe sources, and writing the Python that took validation from 21 hours to 13.

This is that discipline, generalized and open-sourced.

License

MIT